

Quantize with Confidence? An Empirical Study of Quantization for Code Generation

Anonymous Author(s)

Abstract—The growing adoption of local inference frameworks such as Ollama has made it increasingly common for developers to run large code models on laptops and other resource-constrained hardware. In these settings, post-training quantization is essential for reducing memory footprint and enabling practical deployment. However, practitioners currently lack clear guidance on how different quantization techniques affect the functional correctness and quality of generated code. In this paper, we empirically investigate the extent to which state-of-the-art quantization methods — including GPTQ, AWQ, QuIP#, AQLM, BitsAndBytes, and GGUF, affect two large code model families representative of the current state of practice: Qwen2.5-Coder and CodeLlama. Using McEval and CoderEval, two comprehensive multilingual benchmarks spanning Python and Java, we evaluate two dimensions of the generated code: (i) functional correctness, measured via pass@1; and (ii) code quality aspects – including maintainability, reliability, security, and structural complexity. We further introduce a novel analysis of quantization robustness under varying prompt complexity – characterized by Shannon entropy and token length, a dimension that, to our knowledge, remains unexplored in prior work. From the achieved results, it emerges that quantization techniques differ meaningfully in their effect on performance and code quality, for instance AQLM consistently matches or exceeds the full-precision baseline, whereas QuIP# exhibits the largest correctness degradation, particularly on complex prompts. Security attributes remain stable across model families, benchmarks, and programming languages, but sensitivity to prompt complexity varies across techniques. Overall, our findings provide practical guidance for selecting quantization strategies when deploying large code models on resource-constrained hardware. More broadly, they highlight the importance of evaluating quantized models beyond functional correctness to account for code quality and sensitivity to prompt complexity.

Index Terms—Code Generation, AI for Software Engineering, Large Code Models, Quantization, Model Compression, Code Quality.

I. INTRODUCTION

In recent years, Software Engineering (SE) has experienced a significant transformation, largely driven by the integration of Artificial Intelligence (AI) techniques into development practices. Among these, Large Language Models (LLMs) have emerged as powerful tools for automating a wide range of SE-related tasks, enabling developers and practitioners to work more efficiently. Collectively referred to as Large Code Models (LCMs), these models have been successfully applied to various aspects of software development automation, including code generation [1, 2], code summarization [3, 4], code translation [5], and test case generation [6], showcasing their versatility and impact across the whole spectrum of activities characterizing the software development lifecycle.

However, the remarkable capabilities of LCMs come at a significant cost. Models with billions of parameters demand substantial computational resources for inference, and their deployment often requires expensive GPU infrastructure that is inaccessible to many practitioners – particularly those in smaller organizations or academic settings who wish to deploy models locally for privacy, customization, or cost reasons [7, 8]. The environmental implications are equally concerning: training and operating these models at scale produces a considerable carbon footprint, raising questions about the long-term sustainability of current development trajectories [9–11].

In this context, *quantization* [12], the process of reducing the numerical precision of model parameters from 16- or 32-bit floating point to compact representations such as 4-bit integers – has emerged as one of the most practical strategies for bridging the gap between model capability and deployment feasibility. Unlike other efficiency techniques such as knowledge distillation [13] or pruning [14], quantization operates directly on pre-trained model weights without requiring retraining, making it especially attractive for practitioners who need to deploy existing models on resource-constrained hardware.

The growing popularity of quantized models reflects this need in practice. Lightweight local inference frameworks such as Ollama [15] now enable developers to run large language models directly on personal machines, and recent analyses report hundreds of thousands of deployed Ollama instances worldwide, highlighting the rapid growth of local LLM deployments outside centralized cloud infrastructures. At the same time, model repositories such as Hugging Face host a rapidly expanding ecosystem of quantized checkpoints; for example, the widely used *TheBloke* repository alone provides thousands of quantized model variants across multiple architectures and quantization formats [16].

While this rapidly growing ecosystem makes quantized models increasingly accessible, it also introduces substantial variability in the techniques used to compress them. As a result, an emerging body of research has begun to investigate how different quantization strategies affect the behavior of large code models and in which scenarios of code generation this behavior drifts compared to its full-precision counterpart. Early investigations into this question have yielded encouraging but incomplete answers. Wei *et al.* [17] demonstrated that 8-bit quantization can preserve code generation performance with improved energy efficiency. Giagnorio *et al.* [18] pushed the frontier further, showing that 4-bit quantization reduces memory by approximately 70% without significant

degradation, establishing 4-bit as the practical operating point. Nyamsuren [19] reinforced this finding across low-resource programming languages. More recently, Afrin *et al.* [20] took the first step beyond functional correctness by examining how quantization affects code quality attributes such as maintainability and structural complexity, finding that AWQ largely preserves these properties.

However, quantization itself is a fundamentally lossy compression: it reduces numerical precision and the model’s representational capacity, both of which support reasoning over syntax, semantics, and task-specific patterns [21]. This suggests that any loss in generated-code quality may be non-uniform, varying with both the quantization technique applied and the complexity of the input being processed. Existing experimental designs, by averaging across techniques and tasks, may therefore obscure such systematic effects rather than reveal their absence.

All prior work has established quantization as a viable compression strategy for code LLMs, though several important gaps remain. First, existing studies evaluate only a small subset of quantization techniques, despite the Post Training Quantization (PTQ) landscape now encompassing multiple algorithmic families with potentially different behaviors.

Second, most evaluations rely solely on $\text{pass}@k$, overlooking code quality aspects such as maintainability and structural complexity [22, 23]. Third, prior work treats benchmark tasks uniformly and does not examine whether quantization effects vary with prompt complexity, which we approximate using observable properties of the input prompt such as token length and information density (*i.e.*, Shannon Entropy [24]), potentially placing different demands on compressed model representations.

To address these gaps, we conduct an empirical study of six widely used weight-only quantization techniques – GPTQ, AWQ, QuIP#, AQLM, BitsAndBytes, and GGUF – applied at 4-bit precision to two large code model families. We evaluate their impact on functional correctness ($\text{pass}@1$) and code quality (maintainability, reliability, security, and structural complexity) across multiple benchmarks and programming languages, and further analyze how robustness varies with prompt complexity, approximated through token length and Shannon entropy.

Our results show that 4-bit quantization largely preserves functional correctness, but the choice of technique introduces meaningful variation. AQLM consistently matches or exceeds the full-precision baseline, whereas QuIP# shows the largest degradation on complex tasks. Code quality effects are limited but observable: security remains stable, AWQ increases maintainability issues in Java, and BitsAndBytes produces the largest degradation in Python complexity metrics. Robustness to prompt complexity is also strongly model-dependent. These findings provide practical guidance for selecting quantization strategies and highlight the importance of evaluating quantized models beyond functional correctness. All artifacts are released to support replication [25].

The remainder of this paper is organized as follows. Sec-

tion II reviews background and related work. Section III describes the study design. Section IV details the implementation and evaluation setup. Section V presents the empirical findings. Section VII discusses threats to validity, and Section VIII concludes the paper.

II. BACKGROUND AND RELATED WORK

A. Efficiency in Large Code Models

The deployment of billion-parameter large code models imposes substantial computational and energy demands, raising concerns about environmental sustainability [9–11], particularly for practitioners deploying models locally for privacy, customization, or cost reasons [7, 8]. Shi *et al.* [26] identified four key areas for improving sustainability—data reduction, model-centric, system-centric, and program-centric approaches—while Shi *et al.* [27] demonstrated that model compression can reduce energy usage by up to 184× and carbon emissions by 157× with negligible effectiveness loss.

Several directions address these challenges. Parameter-Efficient Fine-Tuning (PEFT) updates only a small parameter subset via adapters [28], LoRA [29], or prompt tuning [30, 31], with promising results on code tasks [32–35]. Knowledge Distillation (KD) trains smaller models to emulate larger ones [36–38], but still requires teacher model access. Pruning removes redundant parameters at the cost of potential performance degradation [14]. Among these, quantization—a model-centric approach [26]—is particularly practical: it directly reduces the precision of pre-trained parameters without retraining, making it well-suited for deploying code LLMs on resource-constrained hardware.

1) *Quantization*: Quantization reduces memory footprint and computational cost by representing model parameters in lower-precision formats (*e.g.*, 4-bit integers) instead of standard 16/32-bit floating point [12, 39]. Quantization-Aware Training (QAT) [40] integrates quantization into training but requires full retraining—prohibitively expensive for billion-parameter LLMs [41, 42]. Post-Training Quantization (PTQ) [43] converts pre-trained models using only a small calibration dataset, making it the dominant approach for compressing large code models and the focus of our study.

We organize weight-only PTQ approaches into six categories (Table I). Four are algorithmic: *second-order/Hessian-informed* methods (GPTQ [44], OWQ [45], SpQR [46]) minimize quantization error using approximate Hessian information; *activation-aware saliency* methods (AWQ [47], SqueezeLLM [48], OmniQuant [49]) protect critical weight channels via activation-magnitude analysis; *rotation/incoherence-based* methods (QuIP [50], QuIP# [51], QuaRot [52], FlatQuant [53]) apply orthogonal transforms to spread error uniformly; and *vector quantization* methods (AQLM [54], VPTQ [55]) encode weight groups through learned codebooks. Two are ecosystem-level: *library/kernel-based* implementations (BitsAndBytes [56], HQQ [57]) provide efficient low-bit kernels with seamless framework integration, and *format/runtime-based* solutions offer

TABLE I
SUMMARY OF POST-TRAINING QUANTIZATION APPROACHES FOR LLMs.
“W” DENOTES WEIGHT-ONLY; “W+A” DENOTES
WEIGHT-AND-ACTIVATION QUANTIZATION. “CODE TASKS?” INDICATES
PRIOR APPLICATION TO CODE-RELATED TASKS.

Approach	Type	Bits	Code Tasks
<i>Second-order / Hessian-informed</i>			
GPTQ [44]	W	2–8	✗
OWQ [45]	W	3, 4	✗
SpQR [46]	W	3–8	✗
<i>Activation-aware saliency</i>			
AWQ [47]	W	4, 8	✓ [20]
SqueezeLLM [48]	W	3, 4	✗
OmniQuant [49]	W / W+A	2–8	✗
<i>Rotation / incoherence-based</i>			
QuIP [50]	W	2–4	✗
QuIP# [51]	W	2–4	✗
QuaRot [52]	W+A	4	✗
FlatQuant [53]	W+A	4	✗
<i>Vector quantization (codebook-based)</i>			
AQLM [54]	W	2–4	✓ [18]
VPTQ [55]	W	2–4	✗
<i>Library / kernel-based</i>			
HQ [57]	W	2–8	✗
BitsAndBytes [56]	W	4, 8	✗
<i>Format / runtime-based</i>			
GGUF	W	2–8	✓ [19]
<i>Weight-and-activation</i>			
ZeroQuant [58]	W+A	4, 8	✗
SmoothQuant [59]	W+A	8	✗
LLM.int8() [60]	W+A	8	✗
RPTQ [61]	W+A	4, 8	✗
PB-LLM [62]	W+A	1, 2	✗

standardized formats for CPU-friendly inference. Weight-and-activation quantization—compressing both weights and activations—is more challenging due to input-dependent outliers; methods such as ZeroQuant [58], SmoothQuant [59], LLM.int8() [60], RPTQ [61], and PB-LLM [62] remain less widely adopted for code tasks. Jin *et al.* [63] corroborated that 4-bit quantization retains performance comparable to full-precision counterparts across instruction-tuned LLMs (7B–72B).

For our study, we select one approach from each category: GPTQ [44] (second-order/Hessian-informed), AWQ [47] (activation-aware saliency), QuIP# [51] (rotation/incoherence-based), AQLM [54] (vector quantization), BitsAndBytes [56] (library/kernel-based), and GGUF (format/runtime-based), ensuring broad coverage across four algorithmic strategies and two dominant deployment pathways, all widely supported in major inference frameworks and with publicly available implementations. Detailed descriptions and configurations are in Section III.

2) *Quantization for Code Models and Code Quality*: While quantization has been extensively studied for general NLP tasks, its application to code-related activities remains comparatively nascent. Wei *et al.* [17] conducted the first large-scale investigation, examining 8-bit quantization on PLBART [64], CodeT5 [65], InCoder [66], and CodeGen [67] for code generation and summarization, finding that quantized models achieve improved energy efficiency with only marginal performance loss. Giagnorio *et al.* [18] extended this to larger models (CodeLlama up to 34B, DeepSeek-Coder up to 33B) using AQLM at 2-bit precision, showing that 4-bit quantization reduces memory by approximately 70% without significant degradation, while more extreme levels (2–3 bits) incur notable

drops that can be partially mitigated through code-specific calibration and post-quantization fine-tuning. Nyamsuren [19] evaluated five 7B models quantized in GGUF format for Lua code generation on consumer hardware, reinforcing the 4-bit trade-off while showing that degradation is more pronounced for low-resource languages. In another work [20] presented the first study to go beyond functional correctness and examine quantization’s impact on code *quality*, finding that AWQ at 4-bit largely preserves quality metrics captured by static analysis tools.

Beyond quantization, a growing body of research recognizes that functional correctness alone is insufficient to characterize the practical utility of generated code [23, 68]. Siddiq and Santos [22] found that LLM-generated code frequently contains quality issues despite being syntactically valid, while Liu *et al.* [69] and Kharmia *et al.* [70] examined iterative refinement and security vulnerabilities, respectively. Yetiştiren *et al.* [23] evaluated code quality across multiple dimensions including maintainability and reliability.

Despite this progress, existing studies have predominantly focused on a single quantization technique per study, with pass@*k* as the primary evaluation criterion. Afrin *et al.* [20] took the first step toward examining quantization’s impact on code quality but were limited to AWQ alone. The present study addresses this gap by comparing six quantization techniques and assessing their differential effects on both functional correctness and code quality.

III. STUDY METHODOLOGY

We evaluate six widely adopted weight-only quantization approaches—GPTQ, AWQ, QuIP#, AQLM, BitsAndBytes, and GGUF—applied at 4-bit precision to Qwen2.5-Coder-7B and CodeLlama-7B, and structure our investigation around the following research questions:

- **RQ₁**: *How do different quantization techniques impact the functional correctness of code generated by large code models?*

In this RQ, we assess functional correctness using the pass@1 metric across two benchmarks (McEval [71] and CoderEval [72]) and two programming languages (Python and Java). By contrasting each quantized variant against its full-precision baseline, we aim to determine whether certain quantization approaches preserve correctness more effectively than others, or whether code generation capabilities degrade uniformly across compression strategies.

- **RQ₂**: *How do different quantization techniques impact the quality attributes of automatically generated code?*

Beyond correctness, RQ₂ examines maintainability, reliability, security, and structural complexity using SonarCloud [73] static analysis. Code that passes test cases may still exhibit excessive complexity or subtle defects that diminish its practical value [22]; we compare quality attributes across all six approaches to identify whether certain methods introduce more pronounced degradation.

- **RQ₃**: *Does input prompt complexity influence quantization-induced correctness degradation?*

We hypothesize that more complex prompts—longer inputs with higher information density—may be disproportionately affected by weight compression. To test this, we evaluate all six approaches on three benchmarks forming a natural complexity gradient: McEval [71] (short, simple prompts), CoderEval [72] (mid-length, context-dependent), and BigCodeBench [74] (long, multi-library orchestration). This analysis is conducted on Python only. We characterize complexity using token length, and Shannon entropy [24], and examine whether correctness degradation correlates with these measures.

The remainder of this section describes our experimental setup, covering the selection of code generation models (Section III-A1), the evaluation benchmarks (Section III-A2), the quantization approaches under investigation (Section III-B), and the static analysis tools employed for quality assessment (Section III-C).

A. Code Models and Dataset

This section outlines the code model families used in our experiments and the benchmarks employed to evaluate code generation across multiple programming languages.

1) *Code Models*: We employ two well-established code model families: Qwen2.5-Coder [75] and CodeLlama [76]. Both have been extensively adopted in code generation and quantization research [18, 20, 77–80]. Following established practices [18, 20, 80, 81], we use the instruction-tuned variant of each model.

Qwen2.5-Coder [75] is a code-specialized adaptation of the Qwen2.5 architecture, pre-trained on over 5.5 trillion tokens comprising 70% code, 20% natural language, and 10% mathematical content [75]. The family spans 0.5B to 32B parameters and has demonstrated strong performance across diverse code tasks [74, 82].

CodeLlama [76] is built on Llama-2 [83] and further trained on 500 billion tokens of natural language and source code. It offers general-purpose and instruction-tuned variants ranging from 7B to 70B parameters.

For both families, we restrict experiments to the **7B variant**. Prior research shows that correctness and quality improve with model size but with diminishing returns [17, 18, 20]; the 7B configuration is representative of the resource-constrained settings where quantization is most commonly applied. Constraining to a single size also keeps the combinatorial experimental space (six approaches $\times$ two benchmarks $\times$ two languages, plus a third benchmark for RQ₃) manageable while enabling deeper cross-technique analysis.

2) *Evaluation Benchmarks*: We employ three code generation benchmarks. **McEval** [71] and **CoderEval** [72] are used in both Python and Java for RQ₁ and RQ₂, and in Python for RQ₃. **BigCodeBench** [74] is used in Python only and contributes to all three RQs, additionally spanning a gradient of input complexity (Python only) in RQ₃.

McEval [71] provides human-annotated, multilingual coding tasks with function signatures, docstrings, and test cases at varying difficulty levels. We use the Python (42 tasks) and Java (53 tasks) subsets. McEval represents the simplest end of our complexity spectrum, with concise function-level prompts.

CoderEval [72] comprises 230 Python and 230 Java tasks from real-world open-source projects, spanning six levels of context dependency—from self-contained functions to project-level contexts. Unlike standalone-only benchmarks, it reflects pragmatic generation scenarios and represents mid-length input complexity with richer contextual information. We use the filtered CoderEval subset of Crupi *et al.* [84] (190 Python and 184 Java tasks), which removes tasks with unreliable test suites. While CoderEval can include file- or project-level context, the natural-language descriptions themselves are typically concise—the contextual code is supplied separately—explaining the relatively low mean prompt length in Table II.

B. Selected Quantization Approaches

As described in Section II-A1, we select six weight-only PTQ approaches—one from each category in our taxonomy: **GPTQ** [44] (second-order/Hessian-informed), **AWQ** [47] (activation-aware saliency), **QuIP#** [51] (rotation/incoherence-based), **AQLM** [54] (vector quantization), **BitsAndBytes** [56] (library/kernel-based), and **GGUF** (format/runtime-based). All models are quantized to 4-bit precision, the most widely adopted operating point for practical LLM deployment [63]. Below, we provide a brief description of each approach and its configuration in our experiments.

GPTQ [44] performs layer-wise weight quantization using approximate second-order (Hessian) information. After each column of the weight matrix is quantized, the remaining weights are adjusted to minimize the overall output reconstruction error. We use 4-bit quantization with a group size of 128.

AWQ [47] identifies salient weight channels by analyzing activation magnitudes and applies per-channel scaling to protect them before quantization. This activation-aware approach preserves the weight channels that contribute most to model accuracy. We apply 4-bit quantization with the default AWQ configuration.

QuIP# [51] uses randomized Hadamard transforms to make the weight matrices incoherent, spreading quantization error uniformly across dimensions, and employs E8 lattice codebooks for efficient encoding.

AQLM [54] employs multi-codebook additive quantization, where groups of weights are jointly encoded through learned codebooks optimized across entire layer blocks. We use the 4-bit configuration with two codebooks.

BitsAndBytes [56] provides efficient low-bit quantization kernels with seamless integration into the Hugging Face ecosystem. We use 4-bit NormalFloat (NF4) quantization with double quantization enabled.

GGUF [85] is a standardized format for distributing quantized model weights, enabling CPU-friendly inference via

the llama.cpp ecosystem. We use the Q4_K_M quantization variant, which applies mixed-precision 4-bit quantization with medium-sized k-quant blocks.

For consistency, we utilize pre-quantized model checkpoints available on Hugging Face where possible, and quantize models locally using official implementations when pre-quantized versions are unavailable.

C. Static Analysis–Based Evaluation of Code Quality and Quantization Robustness

Assessing the quality of generated code spans multiple dimensions—syntax validity, coding style, code smells, reliability, maintainability, and security [23, 68]. While various static analysis tools target these attributes, not all are necessary when their outputs converge on consistent findings.

For this study, we adopt **SonarCloud** [73] as the sole static analysis tool for both Python and Java. SonarCloud identifies bugs, code smells, security vulnerabilities, and code duplications within a single, language-agnostic platform [70]—well suited to our cross-language evaluation.

Rather than supplementing SonarCloud with language-specific tools—such as Pylint [86] and Flake8 [87] for Python or PMD [88] for Java—we adopt a consolidated strategy. This is grounded in empirical evidence from Afrin *et al.* [20], who found strong consistency between quality trends reported by SonarCloud and those captured by Pylint, Flake8, and PMD, with no meaningful divergence in conclusions. Given this alignment, additional tools would contribute redundancy without different insights.

To better understand how robustly quantized models maintain their code generation capabilities as task demands increase, we complement the quality assessment with an analysis of *quantization robustness* across input complexity levels. We characterize each benchmark’s prompts using token length, and Shannon entropy [24]—two complementary, content-agnostic measures motivated by prior code-LLM evaluation work [89] and by NLP literature on entropy as a measure of textual information density [90]—and examine whether quantization-induced correctness degradation, measured as the pass@1 difference between full-precision and quantized models—correlates with these complexity measures.

IV. IMPLEMENTATION AND EVALUATION

A. Experimental Setup and Environment

For each model family (Qwen2.5-Coder-7B-Instruct and CodeLlama-7B-Instruct), we prepare seven configurations: one full-precision (FP16) baseline and six 4-bit quantized variants (GPTQ, AWQ, QuIP#, AQLM, BitsAndBytes, GGUF). Each configuration generates solutions for all benchmark tasks across the three RQs—McEval and CoderEval are used in both Python and Java for RQ₁ and RQ₂ and in Python for RQ₃; BigCodeBench is used in Python only and contributes to all three RQs. Generated code is evaluated for functional correctness (pass@1 [91]) within a sandboxed Docker environment and for code quality via SonarCloud metrics. We set temperature

to 0 for deterministic outputs and cap input tokens at 1024, consistent with related work [92, 93]. All experiments ran on Ubuntu 22.04.5 LTS with four NVIDIA L40S GPUs (48GB each).

B. Model Quantization

For AWQ, GPTQ, GGUF, and BitsAndBytes, we used pre-quantized checkpoints from Hugging Face. For AQLM and QuIP#, we performed quantization using official pipelines with WikiText-2 [94] as the calibration corpus, a standard choice for PTQ [44, 47, 54]. QuIP# quantization used the QuIP-for-all framework [95] to support Qwen-based architectures. All models were loaded via Hugging Face `transformers` to ensure a standardized inference framework.

C. Evaluation Metrics

Functional correctness. We use pass@1 [91], which evaluates whether the model’s top-ranked output passes all unit tests a strict single-attempt measure reflecting realistic deployment.

Code quality. We apply SonarCloud [73] to assess five quality indicators. *Reliability* quantifies code robustness by measuring bug density. *Maintainability* captures code smells, suboptimal patterns that increase technical debt. *Lines of Code (LoC)* counts non-whitespace lines as a proxy for implementation effort. *Cyclomatic Complexity (CyC)* measures structural complexity via the control flow graph ($M = E + 2Q - N$, where E , N , and Q denote edges, nodes, and connected components), with higher values indicating more branching paths. *Cognitive Complexity (CoC)* captures human-perceived difficulty by accounting for nested control flow and conditional depth [96].

D. Input Complexity Analysis (RQ₃)

To investigate whether prompt complexity modulates quantization-induced degradation, we analyze all Python tasks across McEval, CoderEval, and BigCodeBench. For each task, we compute *token length* (word count) and *Shannon entropy* [24] at the word level, capturing input size and lexical diversity respectively.

We pool all tasks and partition them into **High** and **Low** entropy buckets at the median, retaining benchmark identity within each bucket. Table II summarizes the distribution: the High bucket is dominated by McEval and BigCodeBench (mean entropy ≈ 6.1), while the Low bucket is dominated by CoderEval (mean entropy ≈ 3.9). This partitioning captures benchmark-level complexity differences while enabling controlled within-bucket comparisons.

We employ three complementary analyses: (1) **McNemar’s test** [97] on 2×2 contingency tables of discordant pairs to test whether quantization significantly alters per-task correctness; (2) **stratified analysis** of pass@1 and degradation rates within each entropy bucket across the three benchmarks [98]; and (3) **point-biserial correlations** [99] between prompt complexity (entropy, length) and degradation outcome, supplemented by Mann–Whitney U tests [100] comparing complexity distributions of degraded vs. non-degraded tasks.

TABLE II
DISTRIBUTION OF TASKS ACROSS ENTROPY BUCKETS. TASKS ARE POOLED FROM MCEVAL, CODEREVAL, AND BIGCODEBENCH (PYTHON) AND SPLIT AT THE MEDIAN SHANNON ENTROPY.

Bucket	Benchmark	Count	Mean Entropy	Std Entropy	Mean Length
High	McEval	30	6.08	0.18	145.60
	CoderEval	6	6.20	0.13	136.67
	BigCodeBench	650	6.11	0.27	122.34
Low	McEval	12	5.30	0.39	79.92
	CoderEval	184	3.86	0.81	23.30
	BigCodeBench	490	5.47	0.21	68.71

E. Analysis and Comparison Framework

We collect generated solutions from both Qwen2.5-Coder-7B-Instruct and CodeLlama-7B-Instruct across all McEval and CoderEval tasks in Python and Java (RQ_1 , RQ_2), and across BigCodeBench tasks in Python (RQ_3). Each model is evaluated under seven configurations: one full-precision baseline and six 4-bit quantized variants. For every configuration, we capture the generated code and evaluate it along both dimensions: functional correctness via pass@1 and code quality via SonarCloud metrics.

To determine whether quantization introduces statistically significant changes, we conduct pairwise comparisons between each quantized variant and its full-precision counterpart. For pass@1 (binary), we apply **McNemar’s test** [97], which evaluates whether the pattern of per-task pass/fail outcomes shifts significantly based on the 2×2 contingency table of discordant pairs. For SonarCloud quality metrics (continuous), we apply the **Wilcoxon signed-rank test** [101], which assesses whether per-task quality score distributions differ significantly between full-precision and quantized variants. All p-values are adjusted via Holm–Bonferroni correction [102] for six pairwise comparisons. Effect sizes are quantified using Cliff’s delta [103], categorized as negligible (N), small (S), medium (M), or large (L). **To account for output variability, we generated ten predictions per instance across all seven configurations of Qwen2.5-Coder-7B-Instruct on McEval-Python. Friedman’s test [104] found no significant run-level differences in pass@1 or any SonarCloud metric, indicating that the reported differences reflect technique effects rather than inference noise.**

V. RESULTS AND DISCUSSION

We present and discuss the results of our empirical study, organized by research question.

RQ_1 : How do different quantization techniques impact the functional correctness of code generated by large code models?

Table III presents the pass@1 scores for both model families on McEval-Java and CoderEval-Java. On McEval-Java, CodeLlama-7B (FP: 0.25) exhibits wide variation across techniques: GPTQ and AQLM both reach 0.34, while QuIP# drops to 0.08—a substantial reduction indicating that this rotation-based approach is particularly detrimental for CodeLlama on this benchmark. AWQ (0.23) and GGUF (0.23) show marginal degradation, while BitsAndBytes preserves the baseline exactly (0.25). For Qwen2.5-Coder-7B (FP: 0.45), the pattern differs considerably: BitsAndBytes achieves the

highest pass@1 at 0.60—a gain of 15 percentage points—followed by AQLM (0.58) and GGUF (0.51). AWQ matches the baseline (0.45), while GPTQ (0.32) and QuIP# (0.32) exhibit meaningful losses.

On CoderEval-Java—which features pragmatic code generation tasks with real-world context dependencies—the quantized variants cluster more tightly around their baselines. For CodeLlama-7B (FP: 0.31), AQLM (0.33) slightly exceeds FP, while the remaining techniques produce pass@1 scores between 0.28 and 0.30. Even QuIP# (0.28), which suffered substantially on McEval, shows only modest decline—suggesting its degradation pattern is benchmark-sensitive rather than uniform. For Qwen2.5-Coder-7B (FP: 0.20), AQLM leads at 0.27, followed by BitsAndBytes (0.26) and GGUF (0.24).

Two cross-cutting observations emerge. First, the impact of quantization is highly technique-dependent: within the same model and benchmark, pass@1 can range from substantial degradation (QuIP# on CodeLlama-McEval) to notable improvement (BitsAndBytes on Qwen-McEval). Second, the effect is model-dependent: techniques that degrade one model may benefit another. For instance, GPTQ improves CodeLlama’s McEval pass@1 by 9 percentage points but reduces Qwen’s by 13.

McNemar’s test (detailed statistical tables are available in our online appendix [25]) confirms that only one Java comparison reaches significance: CodeLlama FP vs. QuIP# on McEval ($p < 0.05$, OR = 19.00). All other pairwise comparisons are non-significant, indicating that despite considerable variation in aggregate scores, task-level correctness patterns are largely preserved under 4-bit quantization.

Table IV presents the Python results across BCB-Python, McEval-Python, and CoderEval-Python. The patterns largely mirror Java. On McEval-Python, BitsAndBytes achieves the highest pass@1 for CodeLlama (0.19 vs. FP 0.17), while GGUF and AQLM both reach 0.43 for Qwen (vs. FP 0.31). On CoderEval-Python, AQLM leads for both models (CodeLlama: 0.26; Qwen: 0.31). The BCB-Python benchmark reveals the starkest contrasts: AQLM achieves the best CodeLlama pass@1 (0.27 vs. FP 0.23), while QuIP# drops to 0.12—roughly halving the baseline. Qwen’s quantized variants preserve correctness more consistently, with AWQ, GPTQ, and AQLM all matching FP (0.41); QuIP# again shows the largest drop (0.32).

The Python statistical analysis reveals more significant findings than Java, concentrated on BCB-Python: CodeLlama QuIP# ($p < 0.05$, OR = 5.10) and BitsAndBytes ($p < 0.05$, OR = 2.05) significantly degrade correctness, while AQLM ($p < 0.05$, OR = 0.62) significantly *improves* it. For Qwen, only QuIP# reaches significance ($p < 0.05$, OR = 2.71). McEval-Python and CoderEval-Python comparisons remain non-significant. Across both languages, AQLM is the most consistently competitive variant, frequently matching or exceeding FP, while QuIP# exhibits the widest performance variance. Statistical significance is concentrated on BCB-Python, suggesting that quantization-induced correctness shifts become more detectable as task complexity increases—a pattern we

TABLE III

FUNCTIONAL CORRECTNESS (PASS@1) AND CODE QUALITY METRICS OF DIFFERENT MODELS BENCHMARKED ON MCEVAL-JAVA AND CODEREVAL-JAVA. FOR QUANTIZED VARIANTS, ▲ INDICATES IMPROVEMENT OVER THE FULL-PRECISION (FP) BASELINE, ▼ INDICATES DEGRADATION, AND ■ INDICATES NO CHANGE. THE HIGHEST PASS@1 PER MODEL-DATASET PAIR IS HIGHLIGHTED IN GREEN. CYC REFERS TO CYCLOMATIC COMPLEXITY, WHILE CoC DENOTES COGNITIVE COMPLEXITY.

Dataset	Model	Precision	PTQ Technique	Pass@1	SonarCloud Metrics					
					LoC	Security	Reliability	Maintainability	CyC	CoC
McEval-Java	CodeLlama-7B	16 bit	FP	0.25	1301	0	9	274	231	193
			AWQ	0.23	1261▲	0	9	328▼	229▲	211▼
		4 bit	GPTQ	0.34	1384▼	0	9	276▼	243▼	202▼
			GGUF	0.23	1348▼	0	9	276▼	242▼	209▼
			BitsAndBytes	0.25	1368▼	0	9	280▼	251▼	251▼
			AQLM	0.34	1418▼	0	9	274	269▼	273▼
	Qwen2.5-Coder-7B	16 bit	QuIP#	0.08	1217▼	0	10▼	250▲	216▲	193
			FP	0.45	1325	0	9	273	243	211
		4 bit	AWQ	0.45	1310▲	0	9	328▼	252▼	234▼
			GPTQ	0.32	1222▲	0	9	272	215▲	164▲
			GGUF	0.51	1299▲	0	9	278▼	234▲	206▲
			BitsAndBytes	0.60	1368▼	0	9	275▼	261▼	252▼
CoderEval-Java	CodeLlama-7B	16 bit	AQLM	0.58	1339▼	0	9	281▼	248▼	227▼
			QuIP#	0.32	1237▲	0	9	275▼	223▲	203▲
	Qwen2.5-Coder-7B	16 bit	FP	0.31	1810	0	0	306	547	455
			AWQ	0.28	1661▲	0	0	290▲	475▲	369▲
		4 bit	GPTQ	0.30	1714▲	0	1▼	311▼	474▲	362▲
			GGUF	0.28	1648▲	0	0	286	486	386
			BitsAndBytes	0.30	1706▲	0	0	287▲	534	497▼
	Qwen2.5-Coder-7B	16 bit	AQLM	0.33	1833▼	0	0	302	535	403
			QuIP#	0.28	2009▼	0	1▼	305	603▼	573▼
		4 bit	FP	0.20	1215	0	0	228	338	294
			AWQ	0.20	1193▲	0	0	201	337	270
			GPTQ	0.18	1257▼	0	0	227	352	335
			GGUF	0.24	1242▼	0	0	224	344	239
	Qwen2.5-Coder-7B	4 bit	BitsAndBytes	0.26	1293▼	0	0	250	355	319
			AQLM	0.27	1211▲	0	0	250	356	289
			QuIP#	0.16	1049▲	0	0	194	302	225

examine in RQ₃.

Summary – RQ₁

At 4-bit precision, quantization largely preserves functional correctness across both languages. AQLM consistently matches or exceeds the FP baseline, while QuIP# is the most degradation-prone—particularly on complex tasks (BCB-Python), where it reaches significance for both models. The remaining techniques cluster close to their baselines, with no significant correctness shifts on simpler benchmarks.

RQ₂: How do different quantization techniques impact the quality attributes of automatically generated code?

Tables III and IV report SonarCloud metrics—LoC, Security, Reliability, Maintainability, Cyclomatic Complexity (CyC), and Cognitive Complexity (CoC)—for all configurations.

Beginning with McEval-Java, we observe a mixed pattern for CodeLlama-7B. AWQ and QuIP# reduce LoC relative to the baseline (1,261 and 1,217 vs. 1,301), while AQLM produces the longest code (1,418). Security remains unchanged across all techniques (0 hotspots), and Reliability is stable at 9 for all variants except QuIP#, which introduces one additional issue. Maintainability shows notable variation: AWQ increases code smells substantially (328 vs. 274 in FP), while QuIP# reduces them (250). For Qwen2.5-Coder-7B, GPTQ stands out as the most quality-preserving technique, improving LoC (1,222 vs. 1,325), CyC (215 vs. 243), and CoC (164 vs. 211). In contrast, AWQ again inflates Maintainability (328 vs. 273)—a pattern consistent across both model families.

On CoderEval-Java, most CodeLlama variants produce fewer quality issues than FP—AWQ, GGUF, and BitsAnd-

Bytes all reduce LoC, Maintainability, and CyC. The exceptions are AQLM and QuIP#, which generate longer code (1,833 and 2,009 vs. 1,810) with higher complexity. For Qwen, QuIP# produces the most compact code across all metrics (LoC: 1,049, Maintainability: 194, CyC: 302, CoC: 225), while BitsAndBytes and AQLM tend to increase complexity. Directional improvements from quantized models likely reflect stochastic variation rather than systematic quality gains.

The Wilcoxon signed-rank test (see online appendix [25]) confirms that most quality differences are non-significant. The clearest finding is AWQ’s effect on Maintainability: both CodeLlama ($d = -0.346$, medium) and Qwen ($d = -0.394$, medium) show significant increases in code smells on McEval-Java, suggesting AWQ’s activation-aware scaling systematically alters patterns flagged by static analysis. On CoderEval-Java, AWQ shows significant LoC, CyC, and CoC differences for CodeLlama, though all with negligible effect sizes. Across all Java comparisons, Security and Reliability remain unchanged, confirming that quantization does not introduce security vulnerabilities or reliability regressions.

Turning to Python, the quality landscape is more differentiated. On McEval-Python, CodeLlama exhibits a pattern reminiscent of Java: most variants reduce LoC relative to FP, Security remains zero, and directional changes in Maintainability are mixed. For Qwen, BitsAndBytes substantially increases complexity (CyC: 223 vs. 150; CoC: 224 vs. 107), while GPTQ reduces both (CyC: 134; CoC: 80). On CoderEval-Python, a notable asymmetry emerges: CodeLlama’s quantized variants generally improve quality relative to FP, while *all six* Qwen variants substantially increase LoC (1,367–1,618 vs. 988), CyC, and CoC—a consistent quality degradation unique to this model-benchmark-language combination. On BCB-

TABLE IV

FUNCTIONAL CORRECTNESS (PASS@1) AND CODE QUALITY METRICS OF DIFFERENT MODELS BENCHMARKED ON BCB-PYTHON, McEVAL-PYTHON, AND CODEREVAL-PYTHON. FOR QUANTIZED VARIANTS, ▲ INDICATES IMPROVEMENT OVER THE FULL-PRECISION (FP) BASELINE, ▼ INDICATES DEGRADATION, AND ◻ INDICATES NO CHANGE. THE HIGHEST PASS@1 PER MODEL-DATASET PAIR IS HIGHLIGHTED IN GREEN. CyC REFERS TO CYCLOMATIC COMPLEXITY, WHILE CoC DENOTES COGNITIVE COMPLEXITY.

Dataset	Model	Precision	PTQ Technique	Pass@1	SonarCloud Metrics					
					LoC	Security	Reliability	Maintainability	CyC	CoC
BCB-Python	CodeLlama-7B	16 bit	FP	0.23	17484	0	22	819	2726	2109
			AWQ	0.22	17414	0	27	788	2848	2273
		4 bit	GPTQ	0.21	17776	0	17	804	2832	2170
			GGUF	0.21	15623	0	17	735	2675	1988
			BitsAndBytes	0.18	16070	0	15	782	2688	2051
			AQLM	0.27	17006	0	32	883	3179	2446
			QuIP#	0.12	15703	0	43	886	2641	1883
	Qwen2.5-Coder-7B	16 bit	FP	0.41	17281	0	21	670	2975	2492
			AWQ	0.41	17184	0	12	631	2905	2409
		4 bit	GPTQ	0.41	17320	0	11	679	2927	2458
			GGUF	0.40	17148	0	22	664	2983	2565
			BitsAndBytes	0.39	17909	0	30	771	3088	2718
			AQLM	0.41	17278	0	19	667	2987	2577
			QuIP#	0.32	16547	0	14	733	2841	2207
McEval-Python	CodeLlama-7B	16 bit	FP	0.17	1025	0	1	64	207	215
			AWQ	0.17	977	0	2	32	201	177
		4 bit	GPTQ	0.14	1007	0	0	88	215	226
			GGUF	0.12	949	0	0	26	194	187
			BitsAndBytes	0.19	925	0	2	51	187	168
			AQLM	0.12	860	0	0	51	194	138
			QuIP#	0.10	1006	0	2	80	198	194
	Qwen2.5-Coder-7B	16 bit	FP	0.31	816	0	0	25	150	107
			AWQ	0.24	817	0	0	26	157	106
		4 bit	GPTQ	0.19	779	0	0	30	134	80
			GGUF	0.43	953	0	0	31	205	193
			BitsAndBytes	0.38	984	0	1	25	223	224
			AQLM	0.43	958	0	1	33	210	212
			QuIP#	0.36	944	0	1	24	194	157
CoderEval-Python	CodeLlama-7B	16 bit	FP	0.24	1378	0	24	100	489	507
			AWQ	0.24	1255	0	5	71	420	392
		4 bit	GPTQ	0.23	1316	0	4	47	442	424
			GGUF	0.23	1365	0	15	102	475	522
			BitsAndBytes	0.22	1195	0	1	91	432	400
			AQLM	0.26	1460	0	4	142	526	560
			QuIP#	0.21	1138	0	9	52	433	416
	Qwen2.5-Coder-7B	16 bit	FP	0.27	988	0	1	39	350	315
			AWQ	0.28	1500	0	23	36	500	495
		4 bit	GPTQ	0.23	1487	0	32	41	488	515
			GGUF	0.26	1541	0	12	33	524	540
			BitsAndBytes	0.27	1618	0	23	47	526	533
			AQLM	0.31	1469	0	5	28	509	498
			QuIP#	0.24	1367	0	23	48	461	428

Python, QuIP# inflates CodeLlama’s Reliability (43 vs. 22) and Maintainability (886 vs. 819) despite reducing LoC; BitsAndBytes shows the largest divergence for Qwen across LoC (17,909 vs. 17,281), Reliability (30 vs. 21), and complexity.

The Python statistical analysis reveals substantially more significant findings. On CoderEval-Python, all six Qwen variants show significant LoC differences (small to medium effects), with most also showing significant CyC and CoC shifts—confirming that this quality degradation is statistically robust. On McEval-Python, BitsAndBytes produces the only *large* effect size in our entire study: CyC ($d = -0.452$, medium) and CoC ($d = -0.487$, large) for Qwen. On BCB-Python, the large sample size enables detection of many significant but overwhelmingly negligible effects. Security remains zero across all Python configurations. Synthesizing across languages, Security is universally unaffected, Maintainability is the most sensitive dimension (AWQ in Java, BitsAndBytes in Python), and Java quality differences are largely non-significant while Python—especially CoderEval and BCB for Qwen—reveals statistically robust quality shifts.

Summary – RQ₂

Quantization at 4-bit generally preserves code quality, with Security unaffected and most effects negligible. AWQ consistently increases Maintainability issues in Java (medium effect), while BitsAndBytes introduces the largest observed effect in Python (CyC/CoC, large effect). CoderEval-Python Qwen2.5-Coder-7B is the most quality-sensitive configuration, with all six techniques showing significant LoC and complexity increases.

RQ₃: Does input prompt complexity influence quantization-induced correctness degradation?

We pool all Python tasks from McEval, CoderEval, and BigCodeBench, split them at the median Shannon entropy into High and Low complexity buckets (Table II), and measure the point-biserial correlation between complexity (entropy and token length) and the binary degradation outcome—*i.e.*, whether a task solved by FP was broken by the quantized variant. Table V presents per-benchmark degradation rates within each bucket alongside the correlation results.

Across both models, CoderEval tasks—short docstring-style prompts dominating the Low bucket—exhibit near-zero degradation. BigCodeBench concentrates the most severe losses, with QuIP# reaching 12.3–15.3% degradation on CodeLlama. For CodeLlama, the High bucket consistently shows higher degradation on McEval (*e.g.*, AWQ: 6.7% vs. 0.0%; AQLM: 16.7% vs. 0.0%), while for Qwen this pattern partially

TABLE V
PER-TECHNIQUE DEGRADATION RATES (%) WITHIN ENTROPY BUCKETS AND POINT-BISERIAL CORRELATION BETWEEN INPUT COMPLEXITY AND QUANTIZATION-INDUCED DEGRADATION. TASKS FROM MCEVAL, CODEREVAL, AND BIGCODEBENCH (PYTHON) ARE POOLED AND SPLIT AT THE MEDIAN SHANNON ENTROPY. CORRELATIONS (r) ARE COMPUTED OVER ALL POOLED TASKS. SIGNIFICANT CORRELATIONS ($p < 0.05$) ARE HIGHLIGHTED IN RED. * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$.

Model	PTQ Technique	Degr. Rate on High-Entropy (%)			Degr. Rate on Low-Entropy (%)			Entropy Corr.		Length Corr.	
		McE	CodE	BCB	McE	CodE	BCB	r	p	r	p
CodeLlama-7B	AWQ	6.7	0.0	5.5	0.0	2.2	4.7	+0.192	***	+0.248	***
	GPTQ	6.7	0.0	4.3	0.0	3.8	4.7	+0.092	0.108	+0.106	0.063
	GGUF	10.0	0.0	5.4	0.0	3.8	8.6	+0.137	*	+0.207	***
	BitsAndBytes	3.3	0.0	7.4	0.0	2.7	11.6	+0.203	***	+0.187	***
	AQLM	16.7	0.0	6.9	0.0	3.8	9.4	+0.177	**	+0.172	**
	QuIP#	13.3	0.0	12.3	0.0	7.1	15.3	+0.325	***	+0.321	***
Qwen2.5-Coder-7B	AWQ	6.7	0.0	5.2	16.7	5.4	4.9	-0.039	0.374	-0.004	0.923
	GPTQ	13.3	0.0	3.7	8.3	8.2	5.9	-0.156	***	-0.086	*
	GGUF	3.3	0.0	5.1	8.3	4.9	5.7	-0.049	0.263	-0.029	0.506
	BitsAndBytes	6.7	0.0	7.8	16.7	5.4	9.8	-0.011	0.807	+0.015	0.734
	AQLM	6.7	0.0	6.3	8.3	4.3	7.5	+0.014	0.741	+0.054	0.213
	QuIP#	6.7	16.7	14.8	16.7	7.6	15.9	+0.061	0.160	+0.095	*

reverses—several techniques degrade *more* on Low-entropy prompts (e.g., AWQ: 16.7% Low vs. 6.7% High).

The correlation analysis quantifies this model-level divergence. For CodeLlama, five of six techniques show significant positive correlations: QuIP# has the strongest signal ($r = +0.325$, $p < 0.001$ for entropy; $r = +0.321$, $p < 0.001$ for length), followed by BitsAndBytes ($r = +0.203$) and AWQ ($r = +0.192$). GPTQ is the sole exception. For Qwen, only two of twelve correlations reach significance, and one runs in the opposite direction—GPTQ shows a negative correlation ($r = -0.156$, $p < 0.001$), meaning simpler prompts degrade more. All other Qwen techniques show near-zero correlations. One possible explanation is that Qwen, trained on a larger and more diverse corpus, develops more redundant internal representations that are more resilient to precision loss—whereas CodeLlama may rely on less redundant weight configurations for complex reasoning. [Since the High-entropy bucket is dominated by BigCodeBench \(Table II\), we recompute the correlation *within BigCodeBench alone*: the model-level divergence persists—five of six CodeLlama techniques retain significance while Qwen remains largely insensitive—confirming the finding is not a benchmark artifact. Full results are in our replication package \[25\].](#)

The BCB-Python quality metrics (Table IV) add a complementary perspective. For CodeLlama, the techniques that most aggressively quantize—QuIP# and AQLM—not only degrade correctness on complex tasks but also inflate Reliability (43 and 32 vs. 22 in FP) and Maintainability (886 and 883 vs. 819), despite producing shorter code. Conversely, GGUF and BitsAndBytes reduce both LoC and Maintainability relative to FP, suggesting that their correctness losses are not accompanied by proportional quality degradation. For Qwen, the quantized variants are more tightly clustered around the baseline, with BitsAndBytes showing the largest divergence (LoC: 17,909 vs. 17,281; Reliability: 30 vs. 21). These patterns indicate that on complex benchmarks, correctness degradation and quality degradation do not always co-occur—a technique may preserve quality while losing correctness, or vice versa.

Summary – RQ₃

Quantization robustness to input complexity is strongly model-dependent. For CodeLlama-7B, prompt complexity significantly correlates with degradation for five of six techniques, with QuIP# showing the strongest sensitivity ($r \approx +0.32$, $p < 0.001$). For Qwen2.5-Coder-7B, almost no significant correlations exist—the model is largely unaffected by input complexity when quantized.

A. Qualitative Example

To complement the aggregate statistics, we provide a qualitative example in our replication package [25], where we show Qwen2.5-Coder-7B’s outputs for a single Python task under all seven configurations (FP + six quantized variants), annotated with the corresponding pass/fail outcome and SonarCloud/Lizard metrics. The example illustrates two patterns observed throughout our results. First, technique-level variation is visible at the source level: the FP baseline produces a structured, multi-step verification routine, whereas QuIP# collapses the logic into a single tautological expression that passes type checks but fails the test suite. Second, several quantized variants (AWQ, GPTQ, GGUF, AQLM) produce concise yet correct alternatives that match or simplify the FP solution—consistent with the technique-dependent trade-offs reported in RQ₁ and RQ₂. Due to space constraints, the full figure is provided in our replication package [25].

VI. IMPLICATIONS OF OUR FINDINGS

A. Implications for Practitioners

Our findings translate into concrete deployment guidance. Across both model families, 4-bit quantization yields a 63–66% VRAM reduction (14.23 GB $\rightarrow$ ~5 GB on Qwen2.5-Coder-7B), making local inference feasible on consumer GPUs without sacrificing aggregate correctness. Technique selection should follow the dominant deployment constraint:

- **Correctness-first:** AQLM is the only technique with a significant pass@1 *improvement* (BCB-Python, OR=0.62), at the cost of higher latency.
- **Throughput-first:** GPTQ leads in throughput (1.65 $\times$ FP16) while preserving correctness.

- **CPU-only:** GGUF remains the practical choice via the `llama.cpp` ecosystem.
- **Footprint-first:** QuIP# achieves the smallest footprint but produces the only significant correctness degradations and the strongest complexity sensitivity ($r \approx +0.32$, CodeLlama). Use only when memory is binding and prompts are short.
- **Quality-sensitive:** AWQ inflates Java code smells (medium effect) and BitsAndBytes produces the largest Python complexity effect (CoC, large)—weigh these against their otherwise strong correctness profiles.

Security attributes remain stable across all configurations.

B. Implications for Researchers

Three findings have broader implications. **First**, the technique-specific variation we report shows that single-technique evaluations cannot surface the trade-offs that emerge in comparative designs. **Second**, the model-dependence of complexity sensitivity (CodeLlama: 5/6 techniques significant; Qwen: largely insensitive) suggests representational redundancy from larger, more diverse pretraining corpora may act as implicit robustness to weight compression—a hypothesis warranting investigation. **Third**, RQ₃ introduces a previously uninvestigated evaluation axis: pass@1 averaged over a benchmark conceals systematic, complexity-correlated degradation that emerges only under stratified analysis.

VII. THREATS TO VALIDITY

Construct validity. We rely on SonarCloud as the sole static analysis tool, justified by prior evidence of strong consistency with language-specific alternatives [20]. Different tools may capture additional quality dimensions. Our complexity characterization (token length, Shannon entropy) represents only two facets of prompt difficulty; other dimensions such as algorithmic complexity may also influence quantization sensitivity.

Internal validity. We chose two model families (Qwen2.5-Coder-7B, CodeLlama-7B) extensively used in prior quantization research [18, 20], though results may differ for other architectures or scales. For quantization, we used pre-quantized Hugging Face checkpoints where available and quantized locally with WikiText-2 calibration for AQLM and QuIP#—a standard choice [44, 47]; code-specific calibration corpora could yield different results. Temperature was set to 0 for deterministic outputs; results under stochastic sampling may differ. We partially mitigated output variability via a ten-run check on Qwen2.5-Coder-7B-Instruct/McEval-Python; Friedman’s test confirmed no significant run-level differences. For RQ₃, the High-entropy bucket is dominated by BigCodeBench; we mitigate this confound through a within-BigCodeBench replication (Section V) that preserves the model-level divergence.

Conclusion validity. We employ McNemar’s test (binary correctness) and Wilcoxon signed-rank test (continuous quality metrics) with Holm-Bonferroni correction for multiple

comparisons, supplemented by Cliff’s delta effect sizes to distinguish statistical from practical significance.

External validity. Our findings are scoped to 7B models, two languages (Python, Java), three benchmarks, and 4-bit precision. Results may differ at other model scales, bit-widths, languages, or task types. RQ₃ is restricted to Python due to BigCodeBench’s language coverage.

VIII. CONCLUSION AND FUTURE WORK

We compared six weight-only quantization techniques—GPTQ, AWQ, QuIP#, AQLM, BitsAndBytes, and GGUF—at 4-bit precision on Qwen2.5-Coder-7B and CodeLlama-7B, evaluating functional correctness, code quality, and robustness to input complexity. Our results show that 4-bit quantization largely preserves pass@1, but the choice of technique matters: AQLM consistently matches or exceeds the full-precision baseline, while QuIP# produces the only significant correctness losses. Security is unaffected across all configurations, but AWQ significantly increases maintainability issues in Java (medium effect), and BitsAndBytes introduces the largest quality degradation on Python complexity metrics (large effect). Quantization robustness to prompt complexity is model-dependent: CodeLlama shows significant complexity–degradation correlations for five of six techniques (r up to $+0.32$, $p < 0.001$), while Qwen is largely insensitive. Unlike prior work [20] that assessed AWQ alone, our six-technique sweep reveals technique-specific trade-offs that single-technique studies cannot surface; the practical and research consequences are discussed in Section VI.

Future work. Several directions follow naturally from our results. First, extending the analysis to other precision levels (2–3 bit) and larger model scales would test whether the technique-specific trade-offs reported here persist or invert. Second, the use of code-specific calibration datasets [18] may further mitigate the qualitative degradations we observe—notably AWQ on Java maintainability. Third, the interaction between weight quantization and inference-time optimizations such as speculative decoding, KV-cache quantization, and structured pruning remains an open question. Finally, the model-dependence of complexity sensitivity invites a controlled study isolating the role of pretraining-corpus diversity in shaping quantization robustness. A replication package containing all generated code, evaluation scripts, and detailed efficiency measurements is available at [25].

REFERENCES

- [1] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago *et al.*, “Competition-level code generation with AlphaCode,” *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [2] A. Mastropaolo, L. Pascarella, E. Guglielmi, M. Ciniselli, S. Scalabrino, R. Oliveto, and G. Bavota, “On the robustness of code generation techniques: An empirical study on github copilot,” in *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 2023, pp. 2149–2160.
- [3] T. Ahmed and P. Devanbu, “Few-shot training LLMs for project-specific code-summarization,” in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–13.

- [4] T. Ahmed, K. S. Pai, P. Devanbu, and E. Barr, "Automatic semantic augmentation of language model prompts (for code summarization)," in *Proceedings of the IEEE/ACM 46th international conference on software engineering*, 2024, pp. 1–13.
- [5] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, "Large language models for software engineering: Survey and open problems," *arXiv preprint arXiv:2310.03533*, 2023.
- [6] M. Tufano, C. Watson, G. Bavota, M. D. Penta, M. White, and D. Poshyvanyk, "Unit test case generation with transformers and focal context," *arXiv preprint arXiv:2009.05617*, 2020.
- [7] C. Novelli, F. Casolari, A. Rotolo, M. Taddeo, and L. Floridi, "Generative AI in EU law: Liability, privacy, intellectual property, and cybersecurity," *arXiv preprint arXiv:2401.07348*, 2024.
- [8] J. Wu, X. Ouyang, H. Chen *et al.*, "Unveiling security, privacy, and ethical concerns of ChatGPT," *Journal of Information and Intelligence*, 2024.
- [9] J. Castaño, S. Martínez-Fernández, X. Franch, and J. Bogner, "Exploring the carbon footprint of Hugging Face's ML models: A repository mining study," in *2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. IEEE, 2023, pp. 1–12.
- [10] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "Carbon emissions and large neural network training," *arXiv preprint arXiv:2104.10350*, 2021.
- [11] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for modern deep learning research," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 09, 2020, pp. 13 693–13 696.
- [12] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," in *Low-Power Computer Vision*. Chapman and Hall/CRC, 2022, pp. 291–326.
- [13] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015. [Online]. Available: <https://arxiv.org/abs/1503.02531>
- [14] G. D'Aloisio, A. Di Marco, A. Di Stasi, and A. Ferrara, "On the compression of natural language models," *arXiv preprint arXiv:2404.09095*, 2024.
- [15] Ollama Project, "Ollama," <https://github.com/ollama/ollama>, 2024, accessed: 2026-05-08.
- [16] TheBloke, "Thebloke – Hugging Face model repository," <https://huggingface.co/TheBloke>, 2023, accessed: 2026-05-08.
- [17] X. Wei, S. K. Gonugondla, S. Wang, W. Ahmad, B. Ray, H. Qian, X. Li, V. Kumar, Z. Wang, Y. Tian *et al.*, "Towards greener yet powerful code generation via quantization: An empirical study," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 224–236.
- [18] A. Giagnorio, A. Mastropaolo, S. Afrin, M. Di Penta, and G. Bavota, "Quantizing large language models for code generation: A differentiated replication," *arXiv preprint arXiv:2503.07103*, 2025.
- [19] E. Nyamsuren, "Evaluating quantized large language models for code generation on low-resource language benchmarks," *Journal of Computer Languages*, p. 101351, 2025.
- [20] S. Afrin, B. Xu, and A. Mastropaolo, "Is quantization a deal-breaker? empirical insights from large code models," in *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2025, pp. 1–13.
- [21] T. Dettmers, R. Svirschevski, V. Egiazarian, D. Kuznedelev, E. Frantar, S. Ashkboos, A. Borzunov, T. Hoefler, and D. Alistarh, "SpQR: A sparse-quantized representation for near-lossless LLM weight compression," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [22] M. L. Siddiq, L. Roney, J. Zhang, and J. C. D. S. Santos, "Quality assessment of chatgpt generated code and their use by developers," in *Proceedings of the 21st International Conference on Mining Software Repositories*, 2024, pp. 152–156.
- [23] B. Yetişiren, I. Özsoy, M. Ayerdem, and E. Tüzün, "Evaluating the code quality of AI-assisted code generation tools: An empirical study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT," *arXiv preprint arXiv:2304.10778*, 2023.
- [24] C. E. Shannon, "A mathematical theory of communication," *The Bell system technical journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [25] "Replication package," <https://github.com/empirical-quant-project/empirical-quantization-study>, 2026.
- [26] J. Shi, Z. Yang, and D. Lo, "Efficient and green large language models for software engineering: Vision and the road ahead," *ACM Transactions on Software Engineering and Methodology*, 2024.
- [27] J. Shi, Z. Yang, H. J. Kang, B. Xu, J. He, and D. Lo, "Greening large language models of code," in *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS)*. ACM, 2024, pp. 129–140.
- [28] N. Houlsby, A. Giurigu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *International Conference on Machine Learning (ICML)*, 2019, pp. 2790–2799.
- [29] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," *The Tenth International Conference on Learning Representations (ICLR)*, 2022.
- [30] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 3045–3059, 2021.
- [31] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, pp. 4582–4597, 2021.
- [32] D. Wang, Z. Tan, R. Chen, and J. Zhang, "One adapter for all programming languages? adapter tuning for code search and summarization," *Proceedings of the IEEE/ACM 45th International Conference on Software Engineering*, 2023.
- [33] M. Weyssow, X. Zhou, K. Kim, D. Lo, and H. Sahraoui, "Exploring parameter-efficient fine-tuning techniques for code generation with large language models," in *arXiv preprint arXiv:2308.10462*, 2023.
- [34] J. Liu, J. Keung, Q. Zhou, and Y. Liao, "An empirical study of parameter-efficient fine-tuning methods for pre-trained code models," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023, pp. 397–408.
- [35] S. Ayupov and S. Ren, "Parameter-efficient fine-tuning for pre-trained code models," in *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, 2022.
- [36] C.-Y. Hsieh, C.-L. Li, C.-K. Yeh, H. Nakhosht, Y. Fujii, A. Ratner, R. Krishna, C.-Y. Lee, and T. Pfister, "Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes," *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 8003–8017, 2023.
- [37] S. Chaudhary, "Code Alpaca: An instruction-following LLaMA model for code generation," *GitHub repository*, 2023.
- [38] Y. Wei, Z. Wang, J. Liu, Y. Ding, and L. Zhang, "Magicoder: Source code is all you need," *arXiv preprint arXiv:2312.02120*, 2023.
- [39] X. Wang, J. Li, Y. Liu, C. Ma, and W. Wang, "A survey on model compression for large language models," *arXiv preprint arXiv:2308.07633*, 2024.
- [40] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, "Learned step size quantization," *arXiv preprint arXiv:1902.08153*, 2020.
- [41] Z. Liu, B. Oguz, C. Zhao, E. Chang, P. Stock, Y. Mehdad, Y. Shi, R. Krishnamoorthi, and V. Chandra, "LLM-QAT: Data-free quantization aware training for large language models," *arXiv preprint arXiv:2305.17888*, 2023.
- [42] X. Shen, P. Zhao, G. Chen, Z. Wang, Y. Lin *et al.*, "EdgeQAT: Entropy and distribution guided quantization-aware training for the acceleration of lightweight LLMs on the edge," *arXiv preprint arXiv:2402.10787*, 2024.
- [43] Y. Cai, Z. Yao, Z. Dong, A. Gholami, M. W. Mahoney, and K. Keutzer, "ZeroQ: A novel zero shot quantization framework," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 13 169–13 178.
- [44] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2022.
- [45] C. Lee, J. Jin, T. Kim, H. Kim, and E. Park, "OWQ: Outlier-aware weight quantization for efficient fine-tuning and inference of large language models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 12, 2024, pp. 13 355–13 364.
- [46] T. Dettmers, R. Svirschevski, V. Egiazarian, D. Kuznedelev, E. Frantar, S. Ashkboos, A. Borzunov, T. Hoefler, and D. Alistarh, "SpQR: A

- sparse-quantized representation for near-lossless LLM weight compression,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [47] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration,” *Proceedings of Machine Learning and Systems*, vol. 6, pp. 87–100, 2024.
 - [48] S. Kim, C. Hooper, A. Gholami, Z. Dong, X. Li, S. Shen, M. W. Mahoney, and K. Keutzer, “SqueezeLLM: Dense-and-sparse quantization,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
 - [49] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. Qiao, and P. Luo, “OmniQuant: Omnidirectionally calibrated quantization for large language models,” in *International Conference on Learning Representations (ICLR)*, 2024.
 - [50] J. Chee, Y. Cai, V. Kuleshov, and C. M. De Sa, “QuIP: 2-bit quantization of large language models with guarantees,” in *Advances in Neural Information Processing Systems*, vol. 36, 2024.
 - [51] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. De Sa, “QuIP#: Even better LLM quantization with hadamard incoherence and lattice codebooks,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, vol. 235. PMLR, 2024, pp. 48 630–48 656.
 - [52] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, and J. Hensman, “QuaRot: Outlier-free 4-bit inference in rotated LLMs,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 100 213–100 240.
 - [53] Y. Sun, R. Liu, H. Bai, H. Bao, K. Zhao, Y. Li, J. Hu, X. Yu, L. Hou, C. Yuan *et al.*, “FlatQuant: Flatness matters for LLM quantization,” *arXiv preprint arXiv:2410.09426*, 2024.
 - [54] V. Egiazarian, A. Panferov, D. Kuznedelev, E. Frantar, A. Babenko, and D. Alistarh, “Extreme compression of large language models via additive quantization,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
 - [55] Y. Liu, J. Wen, Y. Wang, S. Ye, L. L. Zhang, T. Cao, C. Li, and M. Yang, “VPTQ: Extreme low-bit vector post-training quantization for large language models,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
 - [56] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 10 088–10 115.
 - [57] H. Badri and A. Shaji, “Half-quadratic quantization of large machine learning models,” November 2023. [Online]. Available: https://mobiusml.github.io/hqq_blog/
 - [58] Z. Yao, R. Y. Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, “ZeroQuant: Efficient and affordable post-training quantization for large-scale transformers,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 27 168–27 183.
 - [59] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “SmoothQuant: Accurate and efficient post-training quantization for large language models,” in *International Conference on Machine Learning (ICML)*. PMLR, 2023, pp. 38 087–38 099.
 - [60] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 30 318–30 332.
 - [61] Z. Yuan, L. Niu, J. Liu, W. Liu, X. Wang, Y. Shang, G. Sun, Q. Wu, J. Wu, and B. Wu, “RPTQ: Reorder-based post-training quantization for large language models,” *arXiv preprint arXiv:2304.01089*, 2023.
 - [62] Y. Shang, Z. Yuan, Q. Wu, and Z. Dong, “PB-LLM: Partially binarized large language models,” in *International Conference on Learning Representations (ICLR)*, 2024.
 - [63] R. Jin, J. Du, W. Huang, W. Liu, J. Luan, B. Wang, and D. Xiong, “A comprehensive evaluation of quantization strategies for large language models,” *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
 - [64] W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, “Unified pre-training for program understanding and generation,” *arXiv preprint arXiv:2103.06333*, 2021.
 - [65] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, “CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation,” *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 8696–8708, 2021.
 - [66] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, W.-t. Yih, L. Zettlemoyer, and M. Lewis, “InCoder: A generative model for code infilling and synthesis,” *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
 - [67] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, “CodeGen: An open large language model for code with multi-turn program synthesis,” in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
 - [68] M. L. Siddiq and J. C. Santos, “Generate and pray: Using salmons to evaluate the security of llm generated code,” *arXiv preprint arXiv:2311.00889*, 2023.
 - [69] Y. Liu, T. Le-Cong, R. Widyasari, C. Tantithamthavorn, L. Li, X.-B. D. Le, and D. Lo, “Refining chatgpt-generated code: Characterizing and mitigating code quality issues,” *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 5, pp. 1–26, 2024.
 - [70] M. Kharm, S. Choi, M. AlKhanafseh, and D. Mohaisen, “Security and quality in llm-generated code: A multi-language, multi-model analysis,” *arXiv preprint arXiv:2502.01853*, 2025.
 - [71] L. Chai, S. Liu, J. Yang, Y. Yin, K. Jin, J. Liu, T. Sun, G. Zhang, C. Ren, H. Guo *et al.*, “McEval: Massively multilingual code evaluation,” in *arXiv preprint arXiv:2406.07436*, 2024.
 - [72] H. Yu, B. Shen, D. Ran, J. Zhang, Q. Zhang, Y. Ma, G. Liang, Y. Li, Q. Wang, and T. Xie, “CoderEval: A benchmark of pragmatic code generation with generative pre-trained models,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, 2024, pp. 1–13.
 - [73] SonarSource, “SonarCloud,” <https://docs.sonarsource.com/sonarqube-cloud/>, 2025, accessed: 2025-03-03.
 - [74] T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul *et al.*, “BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions,” *arXiv preprint arXiv:2406.15877*, 2024.
 - [75] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Dang *et al.*, “Qwen2.5-Coder technical report,” *arXiv preprint arXiv:2409.12186*, 2024.
 - [76] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, “Code Llama: Open foundation models for code,” *arXiv preprint arXiv:2308.12950*, 2023.
 - [77] J. Li, G. Li, Y. Li, and Z. Jin, “Structured chain-of-thought prompting for code generation,” *ACM Transactions on Software Engineering and Methodology*, 2023.
 - [78] T. Coignion, C. Quinton, and R. Rouvoy, “A performance study of LLM-generated code on LeetCode,” in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, 2024, pp. 79–89.
 - [79] H. Ren, M. Zhan, Z. Wu, A. Zhou, J. Pan, and H. Li, “Reflection-Coder: Learning from reflection sequence for enhanced one-off code generation,” *arXiv preprint arXiv:2405.17057*, 2024.
 - [80] S. Afrin, J. Call, K.-N. Nguyen, O. Chaparro, and A. Mastropaolo, “Resource-efficient & effective code summarization,” *arXiv preprint arXiv:2502.03617*, 2025.
 - [81] K. Li, Q. Hu, X. Zhao, H. Chen, Y. Xie, T. Liu, Q. Xie, and J. He, “InstructCoder: Instruction tuning large language models for code editing,” *arXiv preprint arXiv:2310.20329*, 2023.
 - [82] S. Quan, J. Ding, Y. Liu, Z. Tang, and H. Ye, “CodeElo: Benchmarking competition-level code generation of LLMs with human-comparable Elo ratings,” *arXiv preprint arXiv:2501.01257*, 2025.
 - [83] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
 - [84] G. Crupi, R. Tufano, A. Velasco, A. Mastropaolo, D. Poshyanyk, and G. Bavota, “On the effectiveness of llm-as-a-judge for code generation and summarization,” *IEEE Transactions on Software Engineering*, 2025.
 - [85] G. Gerganov, “GGUF format specification,” <https://github.com/ggerganov/ggml/blob/master/docs/gguf.md>, 2023, accessed: 2026-05-08.
 - [86] PylintTeam, “Pylint - code analysis for python,” <https://www.pylint.org/>, accessed: 2025-03-03.
 - [87] T. Ziadé and I. Cordasco, “Flake8: Your tool for style guide enforcement. 2021,” URL: <http://flake8.pycqa.org> (besucht am 27. 05. 2019).
 - [88] P. D. Team, “Pmd - source code analyzer,” 2025, static code analysis tool for Java and other languages. [Online]. Available: <https://pmd.github.io>

- [89] Z. Liu, Y. Tang, X. Luo, Y. Zhou, and L. F. Zhang, “Where do large language models fail when generating code?” *arXiv preprint arXiv:2406.08731*, 2024.
- [90] D. Genzel and E. Charniak, “Entropy rate constancy in text,” in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2002, pp. 199–206.
- [91] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [92] J. Wang and Y. Chen, “A review on code generation with LLMs: Application and evaluation,” in *2023 IEEE International Conference on Medical Artificial Intelligence (MedAI)*. IEEE, 2023, pp. 284–289.
- [93] S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty, and S. K. Lahiri, “LLM-based test-driven interactive code generation: User study and empirical evaluation,” *IEEE Transactions on Software Engineering*, 2024.
- [94] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” *arXiv preprint arXiv:1609.07843*, 2016.
- [95] T. Chu, “QuIP-for-all: Unified QuIP# implementation supporting diverse architectures,” <https://github.com/chu-tianxiang/QuIP-for-all>, 2024, accessed: 2026-05-08.
- [96] M. Muñoz Barón, M. Wyrich, and S. Wagner, “An empirical validation of cognitive complexity as a measure of source code understandability,” *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pp. 1–12, 2020.
- [97] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [98] W. G. Cochran, *Some Methods for Strengthening the Common χ^2 Tests*, 1954, vol. 10, no. 4.
- [99] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Lawrence Erlbaum Associates, 1988.
- [100] H. B. Mann and D. R. Whitney, “On a test of whether one of two random variables is stochastically larger than the other,” *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.
- [101] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [102] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [103] R. J. Grissom and J. J. Kim, *Effect Sizes for Research: A Broad Practical Approach*, 2nd ed. Lawrence Erlbaum Associates, 2005.
- [104] M. Friedman, “The use of ranks to avoid the assumption of normality implicit in the analysis of variance,” *Journal of the American Statistical Association*, vol. 32, no. 200, pp. 675–701, 1937.